

Mobile UX & Input

Kursüberblick

- 
1. Intro
 2. Mobile UX & Input
 3. Performance & Mobile Constraints
 4. GPS & Location-Based Mechanics
 5. Map Integration
 6. AR Foundation
 7. Game Loop & Systems
 8. Polishing + Smartphone Features
 9. Prüfung

Heute - Theorie

User Interface

- Landscape vs Portrait
- Safe Areas
- Resolution Scaling

Mobile Input

- Basics
- Unity Input System
- Touch vs Multi Touch
- Gestensteuerung

Heute - Praxis

Taschenmonster-Viewer:

- Creature anschauen
- Creature drehen (horizontal)
- Reinzoomen / rauszoomen

UI - Landscape vs Portrait

Portrait

- gut für Menüs
- Social Apps
- einhändig bedienbar

Landscape

- breiter Bildschirm
- ähnlicher zu PC Games
- besser für Gesten

UI - Landscape vs Portrait

Es geht auch beides

- benötigt Design für beide Varianten
- schränkt uns bei den Mechanics ein
- aber ggf. bessere Erfahrung für den Spieler (z.B. Einhandbedienung)

Für unser Projekt: **fixed Landscape**

- Wer weiß noch wo das eingestellt wird?
- PlayerSettings -> Allowed Orientations for Auto Rotation
 - Auf Auto Rotation lassen
 - **Haken raus bei den Portrait Optionen!**

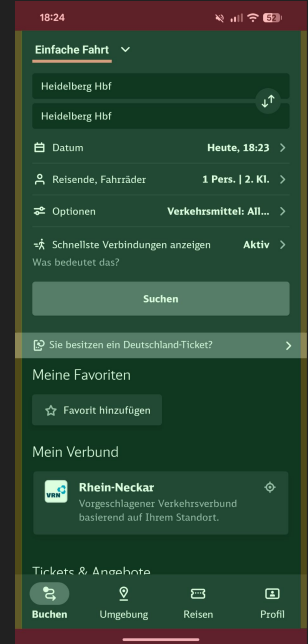
UI - Safe Areas

Nicht der ganze Screen ist nutzbar

- Frontkamera, Statusleiste, abgerundete Ecken, Buttons

Dort dürfen wir keine wichtigen Elemente platzieren

Im Simulator überprüfen ->



UI - Resolutions

Verschiedene Geräte haben verschiedene Auflösungen

- wenn wir UI Elemente absolut beschreiben, haben sie je nach Gerät unterschiedliche Größen
- z.B.: Button mit 200px mal 100px ist auf 1920x1080 Bildschirm deutlich kleiner als auf einem 640x480 Bildschirm.
 - Windows Auflösung runter stellen, alles wird riesig!

Also müssen wir dynamisch arbeiten!

Gibt's da nicht was von Unity?!

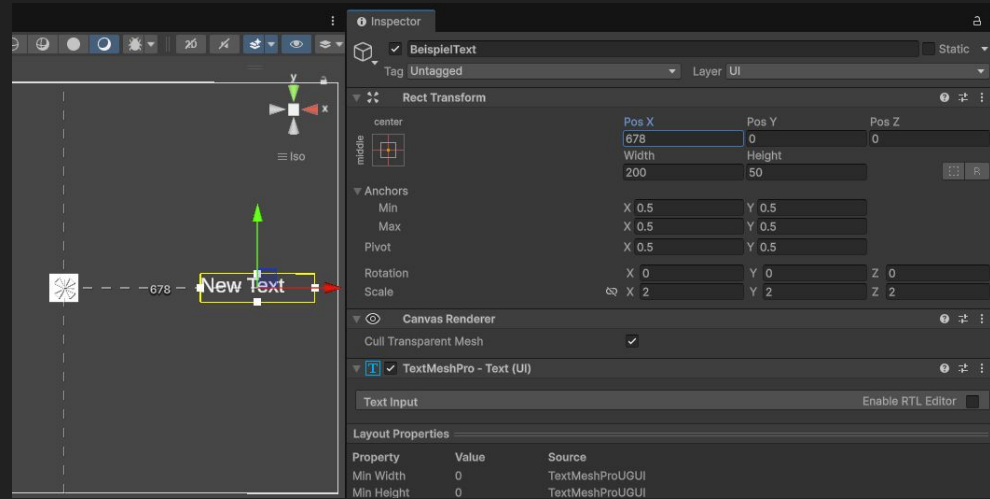
- Na klar!

UI - Canvas

Unity hat das **Canvas**

Ein Canvas ist die Leinwand.

- Auf diese Leinwand “kleben” wir unsere UI Elemente
- platzieren sie **relativ zur Leinwand** selber
- auch die Größe ist **relativ zur Leinwand**

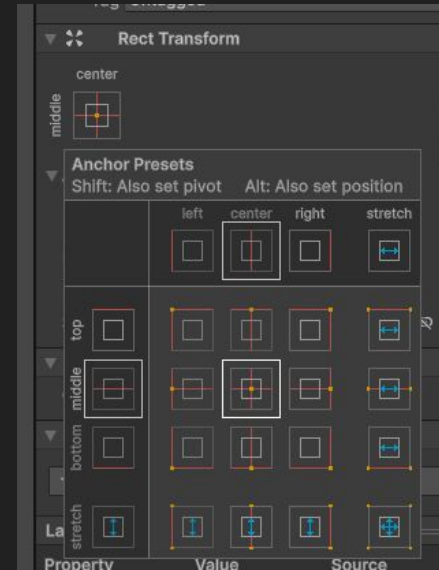
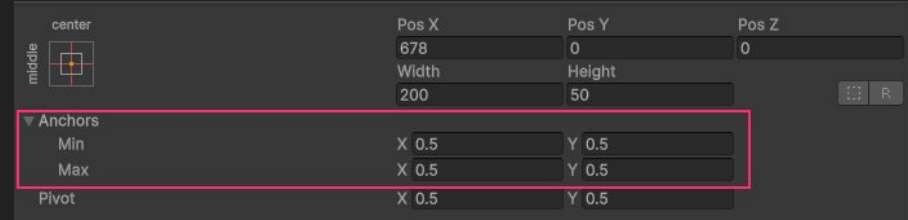


Der Text ist platziert auf **x+678**, **y+0** vom Zentrum der Leinwand

UI - Canvas Anchor

Die Canvas Anchor definieren den Punkt, an den das UI Element “geklebt” wird.

Es gibt verschiedene Presets, die ausgewählt werden können.



UI - Canvas Intermezzo

Aufgabe:

- Canvas erstellen
- Button (keine Logik!)
 - 50x50,
 - mit Text "X"
 - oben rechts geankert, passgenau in die Ecke
- Text
 - 200x50
 - mit Text "Überschrift"
 - oben-mittig geankert, direkt am Bildschirmrand, nicht überstehend!
- Image gelb
 - 100x100
 - oben-links geankert, passgenau in die Ecke
- Image grün
 - 300x200
 - genau in der Mitte geankert

Zeit: 5 Minuten

UI - Canvas Scaler

Und jetzt die Magie!

Canvas Scaler einstellen (Auf Canvas Object)

- Scale Mode: Scale With Screen Size
- Reference Resolution: 1920x1080
- Match = 0.5

Simulator anschauen

- verschiedene **Geräte durchklicken** und dabei **auf die Größe des Canvas achten**

Es skaliert! Allerdings haben wir nicht auf die Safe Areas geachtet!

Mobile Input - Basics

Was sind die größten Unterschiede zwischen PC und Mobile-Input?

Bildschirm

- Display ist auch Eingabegerät!
- Finger verdecken Teile des Screens

Tastatur / Gamepad

- existiert nicht, (theoretisch möglich, aber nicht Standard)

Cursor

- existiert nicht
- Paradigmenwechsel!

Mobile Input - Paradigmenwechsel

PC: Indirekte Interaktion

1. Cursor bewegen
2. Ziel auswählen (Hover)
3. Aktion ausführen (Klick)

Mobile: Direkte Interaktion

1. Objekt berühren = Objekt Reagiert sofort

Wir müssen die Interaktion sofort interpretieren!

Mobile Input - Unity Input System

Früher:

1. Code muss die **Interaktion** auslesen
2. Code reagiert auf die **Interaktion**

```
void Update() // jedes Frame
{
    if (Input.GetMouseButtonDown(0)) // Wurde linke Maustaste gedrückt?
    {
        TriggerFire(); // Löse Feuer Funktion aus
    }
}

void TriggerFire() // Feuer Funktion
{
    Debug.Log("Fire!");
}
```

Mobile Input - Unity Input System

Mit Unity Input System:

1. System erkennt die Interaktionen
2. Verknüpft die Interaktionen mit “Actions”
3. Feuert **Events** sobald die “Actions” ausgelöst werden
4. Script “lauscht” auf das **Event** und reagiert

Mobile Input - Unity Input System

```
void OnEnable() // beim Aktivieren des Objekts
{
    fireAction.Enable(); // Aktiviere die "Action"
    fireAction.performed += OnFireAction; // Verknüpfe "Action wurde ausgeführt"-Event mit Event Listener
}

void OnFireAction(InputAction.CallbackContext context) // EventListener für die Feuer-Action
{
    TriggerFire();
}

void TriggerFire() // Feuer Funktion
{
    Debug.Log("Fire!");
}
```

Komplizierter soll jetzt besser sein??

Mobile Input - Unity Input System

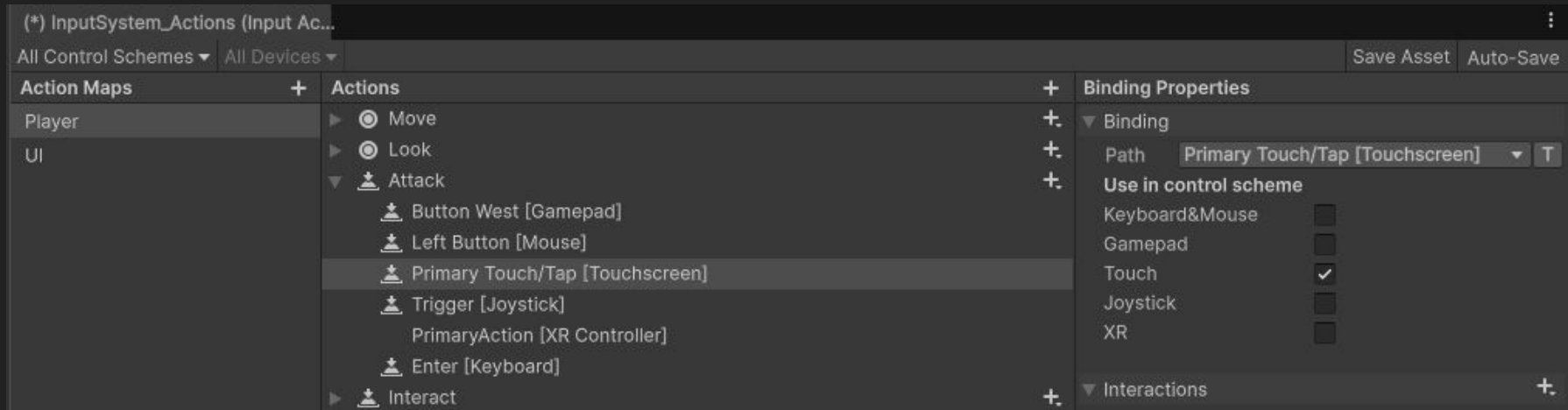
Vorteil:

- Wir müssen die eigentliche Interaktion (Mouse-Click / Button-Press / Touch) nicht kennen.
- Unser Script wartet nur auf den EventTrigger, müssen nicht mehr in jedem Frame das Eingabegerät abfragen
- Wir können die Interaktionen beliebig austauschen, ohne Code zu verändern
- Multi-Platform & Konfigurierbarkeit durch Spieler

Mobile Input - Unity Input System

- Definition der Actions im **InputSystem_Actions Asset**

Machen wir im Praxis-Teil gemeinsam!



Mobile Input - Touch vs Multi Touch

Der Touchscreen kann

- Anzahl der Finger erkennen
- jeden Finger separat verfolgen
- Position und Bewegung messen

Wozu brauchen wir Multi-Touch?

- Mehr als einen Finger erkennen
- zwei Aktionen gleichzeitig ausführen (z.B. Laufen & Schießen)

Mobile Input - Touch vs Multi Touch

Mit Touch und Multi Touch können wir erkennen:

- Wird gedrückt?
- Wo gedrückt?
- Wie lange gedrückt?
- Wieviele Finger drücken?

Damit können wir abdecken:

- Button Drücken
- Objekt Auswählen
- Menü bedienen

Mobile Input - Touch vs Multi Touch

Für den Creature Viewer brauchen wir:

1. Creature anschauen
2. Creature drehen (horizontal)
3. Reinzoomen / rauszoomen

Drehen und zoomen per Button?

Klar geht. Aber da gibt es was besseres:

Gestensteuerung!

Mobile Input - Gestensteuerung

Komplexere Interaktionen intuitiv erledigen!

Reminder: Der Touchscreen kann

- **Position und Bewegung messen** -> Bewegung des Fingers über den Screen verfolgen

Mobile Input - Gestensteuerung - Creature Drehen

Creature drehen mit Tap & Drag

Was müssen wir prüfen?

1. Sind wir im Drag Mode?
 - a. wird mit einem Finger gedrückt?
2. In welche Richtung drehen wir? Links oder Rechts?
 - a. in welche Richtung hat sich der Finger bewegt?
3. Wie schnell drehen wir?
 - a. Wie weit hat sich der Finger bewegt?

Mobile Input - Gestensteuerung - Creature Drehen

1. jedes Frame: Sind wir im "Drag Mode?"

- Beim Starten bool setzen: `isDragging = true;`
- In Update `if(isDragging) HandleDrag();`

2. In welche Richtung drehen wir?

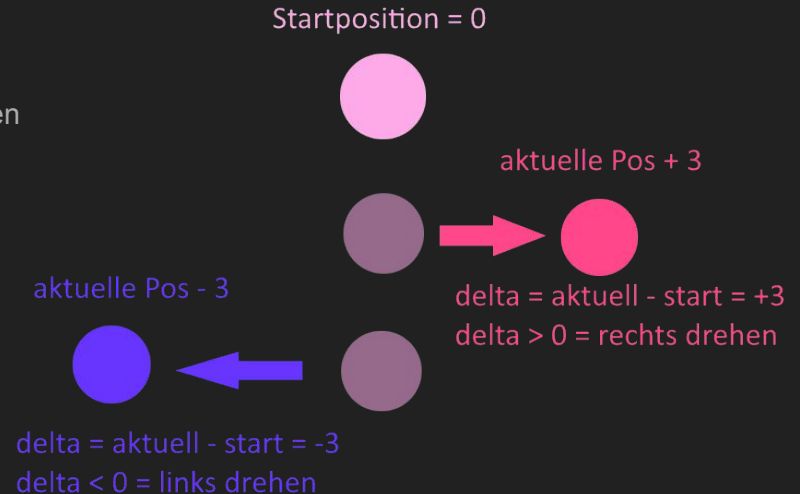
- Speichern der Startposition -> auf nächstes Frame warten
- $\text{delta} = \text{aktuelle Position} - \text{Startposition}$,
- if delta größer als 0
 - rechts drehen
 - else links drehen

3. Wie schnell drehen wir?

- $\text{delta} = \text{faktor für die Drehgeschwindigkeit}$
- weiter bewegt -> größeres Delta
- größeres Delta -> stärker rotieren

4. Rotation anwenden

- `creature.transform.eulerAngles += new Vector3(0, rotationSpeed * delta, 0);`



Mobile Input - Gestensteuerung - Zoom

Reinzoomen / rauszoomen

- Zwei Finger auseinanderziehen / zusammenziehen

Was müssen wir prüfen?

1. Sind wir im Zoom Mode?
 - a. wird mit zwei Fingern gedrückt?
2. In welche Richtung zoomen wir? Rein oder Raus?
 - a. Messen des Abstands zwischen den zwei Fingern
 - b. hat er sich verändert? Und wenn ja in welche Richtung?
3. Wie schnell zoomen wir?
 - a. Wie weit hat haben sich der Abstand verändert?

Mobile Input - Gestensteuerung - Zoom

1. jedes Frame: Sind wir im "Zoom Mode?"

- Beim Starten bool setzen: `isZooming = true;`
- In Update `if(isZooming) HandleZoom();`

2. In welche Richtung zoomen wir?

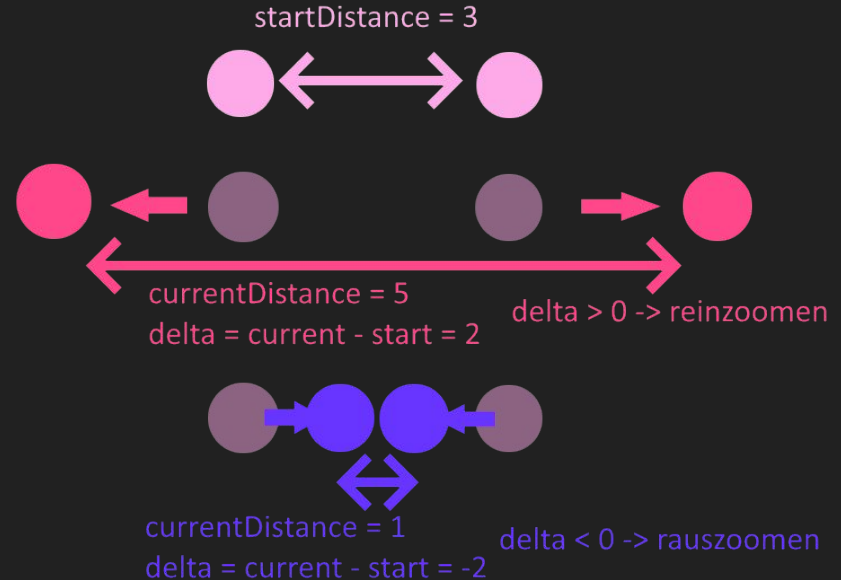
- Fingerabstand speichern:
 - `startDistance = Distance(finger1, finger2);`
- nächstes Frame
- `currentDistance = Distance(finger1, finger2);`
- `delta = currentDistance - startDistance;`
- `delta > 0 -> reinzoomen, delta < 0 -> rauszoomen`

3. Wie stark zoomen wir?

- `zoomSpeed = Math.Abs(delta);`
- Absolute Wert von delta ist unsere Zoomstärke

4. Zoom anwenden

- `creature.transform.localScale *= delta * zoomSpeed;`



Intermezzo - Monster aussuchen & runterladen

<https://github.com/Tillomaticus/MobileGameProg26Assets/tree/2-UI>

Ultimate Monsters



Cube Pets



Mobile Input - Praktisch

Short Demo:

Wie legt man Unity Actions an?

Wie verknüpft man die in einem Script?

Input Actions - Editor

1. Nach Input System suchen
2. Umbenennen in “**PlayerInputSystem**”
 - a. dann Haken setzen bei “**Generate C# Class**” im Inspector
 - b. anschließend öffnen
3. Action Maps -> “Player” und “UI”
4. Daneben “Actions”
 - a. Neue Action anlegen -> “Debug”
 - b. Neues Binding anlegen: (mit Eingabeevent verknüpfen)
 - i. Debug für Editor: Tastatur -> Control Scheme “Keyboard & Mouse”
 - ii. Path anklicken -> Keyboard -> Mapping -> Taste auswählen (z.B. “k”)
 - iii. alternativ: Listen drücken & Taste drücken
5. Save Asset klicken (wichtig!)

Input Actions - Script

1. Namespace des Input Systems hinzufügen
2. PlayerInputSystem-Klasse hinzufügen: `PlayerInputSystem _playerInput;`
(automatisch generierte C# Klasse zu unserem Input-System)
3. in Awake: PlayerInputSystem instanzieren (`new PlayerInputSystem()`)
4. Debug Funktion schreiben (CallbackContext als Funktionsparameter!)
5. In OnEnable:
 - a. InputSystem enablen
 - b. Debug Input finden -> `_playerInput.Player.Debug`
 - c. Callback hinzufügen. -> `+= OnDebugAction;`
 - d. -> In OnDisable beides wieder weg

Input Actions - Testing

Zurück in den Editor und testen!

(Play-Mode Fenster im Fokus!!)

Input Actions - Touch

Geht das auch mit Touch ? - Na klar!

Reihenfolge wieder:

1. Action anlegen
2. Script schreiben
3. Testen

Also wieder rein ins Input Asset

Input Actions - Touch Actions anlegen

1. Neue Action Map erstellen -> "Touch"
2. Neue Action "PrimaryTouch"
 - a. neues Binding -> Touchscreen -> Primary Touch -> Touch Contact?
3. Neue Action "PrimaryTouchPosition"
 4. ActionType ändern in "Value"
 5. neues Binding -> Touchscreen -> Primary Touch -> Position
6. Save Asset!

Input Actions - Touch Action Script

1. Wieder `PlayerInputSystem` anlegen und aktivieren
2. Diesmal statt `Player.Debug` auf `Touch.PrimaryTouch.performed`
 - a. gibt uns das "Durchgeführt" Event.

Und was ist wenn der Finger drauf bleibt?

1. neue Funktionen für Start und End:
 - a. `Touch.PrimaryTouch.started` & `Touch.PrimaryTouch.canceled`

Testen

Input Actions - Touch Position Script

Start und End werden erkannt. Und wie kriegen wir die Position?

1. HandleDrag Funktion

- `_playerInput.Touch.PrimaryTouchPosition.ReadValue<Vector2>();`
- Wert per `Debug.Log` ausgeben

```
1 reference
void HandleDrag()
{
    // Auslesen der FingerPosition via Vector2 Variable"
    Debug.Log("Finger at position " + _playerInput.Touch.PrimaryTouchPosition.ReadValue<Vector2>());
}
```

- isDragging bool anlegen
- HandleDrag in Update() aufrufen, wenn "isDragging = true"
- In Start und End Funktionen "isDragging"-Flag setzen

Kurze Pause?

Aufgabe: Taschenmonster-Viewer

Funktionen:

1. Über UI-Buttons 3 Creatures auswählen und anschauen können
 - a. 3 Buttons machen (Canvas! Canvas-Scaling!)
 - b. Umschalten der Creatures per Click (CreatureViewer-Script)
2. Creature drehen (horizontal)
 - a. Drag Geste erkennen
 - b. Creature Rotation mit Drag Geste verknüpfen
3. Reinzoomen / rauszoomen
 - a. Pinch Geste erkennen
 - b. Camera Size mit Pinch Geste verknüpfen

Tipps:

1. Für die Buttons reicht das “OnClick()” Event im Button Inspector -> kein extra Script!
 - a. Von da aus einfach das CreatureViewer Script aufrufen um die Creatures umzuschalten
2. CreatureViewer Script mit dem InputSystem aus der Touch Demo erweitern!
 - a. Delta der xPosition zwischen alter und aktueller TouchPosition ermitteln
 - b. Delta an Creature Rotation knüpfen (**transform.Rotate**)
3. Zoom Funktion
 - a. InputActions: **Touch#1/TouchContact?** & **Touch#1/Position**
 - b. selbe Ablauf wie bei PrimaryTouch
 - c. Distanz ermitteln: **Vector2.Distance(firstFinger, secondFinger)**
 - d. **Camera.orthographicSize** um **delta*zoomSpeed** erhöhen / verringern
 - i. *Tipp : (die Min / Max Werte für den Zoom angeben:* `newSize = Mathf.Clamp(newSize, _minMaxZoom.x, _minMaxZoom.y);`

Häufige Fehler:

1. Rotation / Zoom “springt” beim neu ansetzen

- beim Start des Touch die `_oldXPosition` auf die aktuelle XPosition setzen
- beim Start des DoppelTouch die `_oldDistance` auf die setzen

```
// zwischenspeichern der Fingerpositionen
Vector2 firstFingerPosition = _playerInput.Touch.PrimaryTouchPosition.ReadValue<Vector2>();
Vector2 secondFingerPosition = _playerInput.Touch.SecondaryTouchPosition.ReadValue<Vector2>();

// überschreiben des Wert für die alte Distanz um ein Snapping zu verhindern, wenn wir den Pinch neu ansetzen.
_oldDistance = Vector2.Distance(firstFingerPosition, secondFingerPosition);
```

2. kleine Werte für die Speed's verwenden

- `rotationSpeed = 0.1`, `zoomSpeed = 0.005`

Bonus:

1. Rotation und Zoom normalisieren (Auflösungs-unabhängig)

- a. Aktuell wird bei der Distanzberechnung mit Pixeln gerechnet. Die hängen aber an der Auflösung. D.h. die Zoom und Dreh-Geschwindigkeit beschleunigen sich mit höheren Auflösungen!
- b. Es gibt aber Möglichkeiten die Distanzberechnung unabhängig von der Auflösung zu machen

Tipp:

- für Rotation -> Normalisieren mit Screen.Width
- für Zoom -> Distanz-ratio statt absoluter Distanz